

Project Report: Ruby as a Modern Programming Language

Duy Hoang

CSIT 313_02

Professor Robila

May 3, 2026

Table of Contents

1. Introduction	3
2. Background and History	3
3. Main Features of Ruby	4
5. Strengths of Ruby	6
6. Limitations	7
7. Comparative Analysis with other languages.....	7
8. Personal Position and Critical Evaluation	8
9. Conclusion.....	8
10. References	9

1. Introduction

The programming languages are based on the needs of programmers and software engineering. Ruby is an unconventional program language, which places an emphasis on human friendly design, reading and writing rather than just performance. Ruby by Yukihiro Matsumoto is a paradigm shift towards user-friendly and satisfactory programming (Heurich, 2024). In this paper a teaching methodology has been adopted to introduce readers to Ruby and evaluate the approach in terms of its appropriateness, advantages and drawbacks in the contemporary software engineering.

This report is explanatory and analytical in elaborating understanding and evaluation as opposed to description only. The main argument presented is that Ruby is not yet going away as a niche language that can still be utilized in rapid web development and prototyping specifically. It does not have a mainstream future or performance scalability. In this report, it is argued that the beauty of Ruby is seen in its efficiency in the task at hand, supported by the current studies in academia and industry.

2. Background and History

Yukihiro Matsumoto created Ruby in the mid 1990s with a very particular purpose: to provide programmers with an enjoyable language in which to write programs. Unlike most other programming languages which are developed based on computational efficiency, system performance and more human factors have been utilized as an important consideration in the development of ruby: simplicity, flexibility, and readability. This method is even commonly called as "Programming happiness" which implies that the language favors minimizing the frustration of developers and maximizing productivity. This is facilitated by the fact that Ruby supports several programming paradigms like object oriented, procedural and functional programming. It implies that the language does not limit developers to a particular paradigm since developers can select a paradigm, which suits their requirements.

One of the critical milestones in the growth of Ruby has been the introduction of Ruby on Rails in 2004 that has made Ruby gain an enormous popularity particularly in web development. This architecture was the first to have used the motto of convention over configuration whereby the developers are not required to explicitly configure all that the framework does, and it has sensible defaults. This simplifies and speeds up the programming process and allows developers to concentrate on the application and not the code. This led to the high adoption of Ruby by start ups and web development.

However, with all the popularities in the beginning, Ruby and especially Rails have lost their leadership to other languages. Research has shown that it has declined because of performance problems and scalability (Klochkov and Mulawka, 2021). The development of Ruby is part of a broader movement of programming language design patterns to machine efficiency to human efficiency. This has democratized the process of software development by rendering the process to be faster and more productive at the expense of speed and efficiency of some performance critical applications.

3. Main Features of Ruby

The most efficient method of looking at the features of the Ruby language is to offer a combination of both explanation and practical example as only in this manner one will not only get to see how the language is supposed to work but also why it is assumed to be this special. Ruby is a pure object oriented language in its most primitive form, i.e. everything in the language itself is an object, even numbers, strings or even classes. This design brings conceptual consistency making developers deal with everything in a consistent manner. To give an example, even this simple program

```
5.times {puts "Hello"}  
Hello  
Hello  
Hello  
Hello  
Hello
```

demonstrates that even such a simple program as that is an object and can call a method. This consistency reduces the ambiguity and makes the language easy to learn particularly. This design choice can, however, introduce a slight performance overhead over languages that are based on primitive data types as everything now has to be handled through object structures.

The other interesting aspect in the Ruby is that it has very readable code. The language was crafted to resemble normal human language and this is a huge leap in making the barrier to entry of new programmers smaller. This readability also leads to the collaboration process since code that is written in Ruby is usually easy to read, examine and maintain. This leads to reduced error rates as well as increased long run maintainability in a professional setting where multiple developers are working on the same codebase. Individuals are likely to praise Ruby as it decreases cognitive

complexity, which allows developers to have more time to resolve a problem rather than spend time trying to understand syntax.

Ruby is also dynamic typed. Variables do not require explicit typing and may dynamically change type. This factor introduces the flexibility and accelerates the development process especially during the initial stages of a project where the requirements are subject to change. Programmers can write at a faster rate without necessarily struggling with fixed type constraints. There are however tradeoffs to this flexibility. Dynamic typing is demonstrated to add run-time errors and makes debugging more difficult in large and complex systems (Akbarpour et al., 2025). Quick development and prototyping Dynamic typing may facilitate it, but in scale, it can compromise reliability unless there is an effective culture of testing.

Ruby provides some other prevailing constructs such as blocks and iterators that enable the statements of codes to be concise and expressive. An example of this is the phrase

```
[1,2,3].each { [n] puts n }
```

Rubi example .rb

1

2

3

teaches us how to step over a bunch of stuff, in a simple and terse way, which Ruby can do. These proxies are used by the object oriented programming language of Ruby so as to support a functional style of programming that can assist developers to write beautiful and efficient programs. This is among the strengths of Ruby as it gives the flexibility to the problem solving methods.

One of the most complex features in Ruby is metaprogramming which enables programs to dynamically create or modify code. This ability is especially essential in the creation of systems like Ruby on rails, in which common work can be automated and generalized. Nevertheless, as much as metaprogramming provides more power and flexibility it may diminish code transparency and thus complicate debugging when carried to an extreme. It needs close and strict usage.

It is fairly easy to begin with Ruby. The Ruby interpreter is typically installed by a developer, who then writes code in .rb files and runs it in a command line interface. Frameworks such as Rails make the process of development even easier by offering ready-to-use structures and tools so that developers can create an application in no time.

Applications Ruby is best known in web application development, specifically with the Ruby on Rails framework allowing web applications to be quickly developed using databases. Research indicates that Rails is still useful in a structured and high-paced development space (Tripathi et al, 2024). Outside web development, Ruby is popular as a scripting and automation language because of its simplicity and the expressive syntax. It enables developers to automate mundane processes like file processing and running systems effectively. It has also found use in DevOps tools, though its position in this field is fading slowly to more performance-focused languages.

Speed of development is more important than optimization, making Ruby especially suitable to rapid prototyping. It is therefore very useful in a startup setting where speed of iteration and speed of time to market are important elements. Industry statistics show that Ruby still accommodates a large number of developers, and is used in production systems, although the overall popularity has decreased (Zipdo, 2026). This means that as much as Ruby may no longer be as dominant in the programming world as it used to be. It still has a major and viable role to play in the current software development.

5. Strengths of Ruby

- The first strength of the language is that the language is productive to work with. It has a reduced syntax, which allows coders to produce advanced functionality with less number of code lines, which reduces the cost of development and increases productivity. It is useful particularly in the startups and agile environment.
- Readability and maintainability is another significant benefit. Ruby has the advantage of simplicity in syntax allowing software developers to more easily analyze and re write the code, unlike in group development projects, and in the long term support of software maintenance.
- Rails ecosystem enhances the capabilities of the Ruby too and it also makes it a web development framework. It is able to simplify operations such as database management, routing, and security and enables application developers to focus on the logic of the application rather than infrastructure.
- The other strength is that of flexibility of Ruby. The developers have the opportunity to tackle the problem in alternative ways that allow creativity and innovation. This flexibility should however be approached with care so that uniformity of teams is maintained.

- There is still the continuing improvement of Ruby. Indicatively, the newer editions of the Ruby language have offered performance improvements like Just in Time (JIT) compilation, which makes the code run much faster.

6. Limitations

- Ruby has a number of serious limitations despite its benefits. The best known challenge is performance. Research suggests that Ruby on Rails applications can be slow to respond when compared to other competing frameworks, especially in large-scale applications (Klochkov and Mulawka, 2021). This is the limitation that limits the usage of Ruby in performance critical applications.
- Scalability is another concern. Scalability in Ruby applications is possible, but sometimes more optimization and infrastructure are required, making it more complex and expensive. This makes Ruby less suitable for high-performance distributed systems.
- Ruby has also lost popularity during the recent years. Studies examining the trends of developers demonstrate that the use of Ruby and its discussions are gradually decreasing, with a shift to other languages (Akbarpour et al., 2025). The drop impacts employment, the development of the community, and sustainable environmental ecosystems.
- Dynamic typing, though advantageous in flexibility, presents risks to large scale applications. It may cause runtime error and make debugging difficult. Excessive metaprogramming can lead to reduction in clarity of code, thus complicating system maintenance.

7. Comparative Analysis with other languages.

The role of Ruby is easier to understand in comparison to other programming languages.

Table 1: Comparative Analysis of Key Features Between Ruby and Python

Feature	Ruby	Python
Syntax	Highly expressive	Simple and structured
Popularity	Declining	Increasing
Performance	Slower	Moderately faster
Use Cases	Web (Rails)	AI, data science

The ability of Python to dominate some of the new areas of artificial intelligence underscores the failure of Ruby to extend to new areas.

Table 2: Comparative Analysis of Ruby and JavaScript Across Development Environment, Ecosystem, and Performance

Feature	Ruby	JavaScript
Environment	Backend focused	Full stack
Ecosystem	Rails	Node.js, React
Performance	Slower	Faster

JavaScript is more multipurpose and has a larger range of applications; whereas Ruby is a specialized language in the field of backend web development.

8. Personal Position and Critical Evaluation

Ruby is a completely different philosophy of programming one that puts human factors first, and machine efficiency second. The philosophy has inspired current programming, even in more technically competent languages. Ruby performs better in the situations, where readability, productivity and quick development are critical. It is especially useful in startups, web application development and in prototyping situations. However, it is less suitable for high performance systems, large-scale distributed architectures, and data intensive applications.

Ruby ought to be viewed as a contextual language, not a declining or outmoding language. Its decrease in popularity is more of a changing of priorities within the industry than an inherent lack of quality. Recent developments in the performance of Ruby and its ecosystem show continued development and relatability.

The future of Ruby is not to fight against the leading languages, but a niche with its strong points that will be of the most use.

9. Conclusion

Ruby is also an important programming language as it is a human centered language. It has an expressive syntax and is well supported by the ecosystem. Although it has its own problems like performance constraints and loss of popularity, it still offers value in certain fields especially web development and fast application development. This report has revealed that Ruby is not outdated but is contextually appropriate. Its productivity and readability can be considered its strong suits since it is bound to be used even as the larger framework of programming continues to develop. The legacy

of Ruby is its focus on developer experience, which remains part of modern software development to this day.

10. References

Akbarpour, N., Mirza, A.S., Raoofian, E., Fard, F.H. and Rodríguez-Pérez, G. (2025) *Unveiling Ruby: Insights from Stack Overflow and Developer Survey*.

Benis, M.S., Akbarpour, N., Fard, F.H., Ouni, A. and Saied, M.A. (2025) *Collaborative Agents for Automated Program Repair in Ruby*.

Chen, F., Fard, F., Lo, D. and Bryksin, T. (2022) *On the Transferability of Pre-trained Language Models for Low-Resource Programming Languages*.

Heurich, G.O., (2024). *Coderspeak: The language of computer programmers*. UCL Press.

Klochkov, D. and Mulawka, J. (2021) *Improving Ruby on Rails-Based Web Application Performance*. *Information*, 12(8), 319.

Tripathi, M., Jain, D., Sharma, K., Anuradha and Daulatram (2024) ‘Web Development Framework’, *International Journal of Advanced Research in Science Communication and Technology*

Zipdo (2026) *Ruby Statistics and Industry Trends Report*.
<https://zipdo.co/transformation-statistics/> (Accessed: March 27 2026)